BANCO DE DADOS NoSQL: FLEXIBILIDADE E DESEMPENHO EM SISTEMAS DE ARMAZENAMENTO NÃO RELACIONAL



DISCIPLINA: BANCOS DE DADOS II

Curso: Engenharia de Software

1 INTRODUÇÃO

Durante mais de quatro décadas, os sistemas gerenciadores de banco de dados relacionais consolidaram-se como a base do armazenamento de informações nas organizações (ELMASRI; NAVATHE, 2018). No entanto, a partir da popularização dos meios digitais, antes muito mais concentrados dentro da academia científica, sobretudo a partir da difusão da internet, houve um aumento enorme na produção de dados digitais pela população. A partir de um estudo realizado ao longo de duas décadas, Hilbert e López (2011) estimaram que a capacidade tecnológica mundial de armazenamento, comunicação e processamento de informações cresceu de forma acentuada entre 1986 e 2007.

Nesse contexto, o volume, a variedade e a velocidade de geração dos dados cresceram em grande escala, levando a criação do conceito de Big Data. Por conta das limitações tecnológicas de bancos de dados tradicionais, é nesse cenário que ganham espaço os bancos de dados NoSQL, sistemas não relacionais que priorizam a flexibilidade do esquema, a escalabilidade horizontal e o desempenho em grandes volumes de dados. Tem-se então que a escolha da tecnologia de armazenamento passou a impactar diretamente o desempenho, o custo e a evolução dos sistemas de informação, que cada vez mais combinam diferentes tecnologias, cada uma adequada a um tipo de acesso aos dados.

Diante disso, este ensaio tem por objetivo discutir os fundamentos dos bancos NoSQL, sua relação com os modelos OLTP e OLAP e suas diferenças em relação aos bancos relacionais.

2 FUNDAMENTOS DE NoSQL

O termo NoSQL significa Not Only SQL, o qual se refere ao conjunto de sistemas que não adotam os modelos relacionais predominantemente. Essa expressão foi empregada em 1998 por Carlo Strozzi para nomear um banco relacional sem SQL, porém sua acepção atual consolidou-se somente em 2009 a partir de um encontro de desenvolvedores dedicado a bancos distribuídos e não relacionais de código aberto. O surgimento dessas soluções está associado às grandes empresas de internet, que precisavam processar volumes massivos de dados com alta disponibilidade. Dentre as grandes vantagens que oferecem, podem serem citadas:

- Flexibilidade de esquema: normalmente permitem armazenar registros com estruturas diferentes, sem exigir um esquema rígido previamente definido. Isso facilita mudanças na aplicação e a inclusão de novos atributos.
- Escalabilidade horizontal: os sistemas NoSQL não são estritamente dependentes de servidores únicos robustos. Foram projetados para distribuir dados e carga entre vários servidores. Isso favorece aplicações de grande escala.
- Bom desempenho em operações de grande volume: alguns sistemas NoSQL conseguem realizar leituras e escritas rapidamente porque os dados podem ser armazenados de acordo com o padrão de acesso da aplicação, evitando junções complexas.

Os bancos de dados NoSQL podem ser classificados em quatro grandes categorias, sendo essa divisão realizada principalmente de acordo com a maneira como os dados são estruturados e armazenados em cada sistema (ELMASRI; NAVATHE, 2018). Entre essas categorias, encontram-se, os bancos de chave-valor, nos quais cada informação é armazenada a partir da associação entre uma chave única e determinado valor. A chave funciona como uma forma de identificar e acessar diretamente aquela informação, o que permite que esse tipo de banco apresente um acesso bastante rápido aos dados. Por esse motivo, eles são muito utilizados, por exemplo, em situações como armazenamento de sessões e *caches.* Existem os bancos orientados a documentos, os quais possuem suas informações armazenadas na forma de documentos geralmente utilizando formatos como JSON ou BSON. Uma das principais vantagens é justamente a possibilidade de armazenar estruturas mais complexas e até mesmo aninhadas dentro de um mesmo documento, sem a necessidade de dividir necessariamente essas informações em diversas tabelas, como normalmente ocorre nos bancos relacionais. O MongoDB é um dos principais exemplos dessa categoria.

Outra categoria são os bancos de famílias de colunas, como o Cassandra. Nesse modelo, as informações são organizadas em grupos, ou famílias, de colunas relacionadas entre si e associadas a uma determinada chave de linha. Esse tipo de organização permite trabalhar de maneira eficiente com grandes volumes de dados e, principalmente, com situações que envolvem um número muito elevado de operações de escrita. Por fim, existem os bancos orientados a grafos, que são estruturados de uma maneira um pouco diferente das categorias anteriores. Estes representam não somente os dados, mas também as relações existentes entre eles. Nesse modelo, as entidades são representadas como nós, enquanto os relacionamentos entre essas entidades são representados por arestas. Dessa forma, esse tipo de banco acaba sendo especialmente interessante para aplicações nas quais os relacionamentos entre os dados possuem grande importância, por exemplo, em redes sociais.

É importante ressaltar que, apesar de apresentarem robustez para aplicações que processam grandes quantidades de dados, os bancos NoSQL implicam em alguns pontos negativos, ou seja, fraqueza dessa família de sistemas:

- Ausência de uma linguagem de consulta padronizada: diferentemente dos bancos relacionais, que utilizam a linguagem SQL como um padrão amplamente estabelecido, os bancos NoSQL não possuem uma linguagem única de consulta. Dessa forma, cada tecnologia pode apresentar sua própria forma de realizar consultas e manipular os dados, o que pode aumentar a dificuldade de aprendizado e também reduzir a portabilidade entre diferentes sistemas.
- Suporte limitado a junções: em muitos bancos NoSQL, as operações de junção entre diferentes conjuntos de dados são inexistentes ou mais limitadas quando comparadas aos bancos relacionais. Isso acontece porque esses sistemas normalmente procuram armazenar os dados de acordo com a forma como eles serão acessados pela aplicação, evitando a necessidade de realizar diversas junções durante uma consulta.

- Duplicação de dados: outra característica comum é a utilização da desnormalização. Nesse caso, uma mesma informação pode ser armazenada em mais de um local com o objetivo de facilitar e tornar mais rápido o acesso aos dados. No entanto, essa duplicação pode aumentar a quantidade de armazenamento necessária e também gerar uma preocupação adicional com a manutenção da consistência dessas informações.

3 RELAÇÃO COM OLTP E OLAP

Quando se analisa a natureza do processamento realizado pelos sistemas de informação, é possível separar essas operações em dois grandes modelos, conhecidos como OLTP (*Online Transaction Processing*) e OLAP (*Online Analytical Processing*). Apesar de ambos trabalharem com dados, eles possuem objetivos diferentes e, consequentemente, apresentam características bastante distintas.O OLTP está relacionado principalmente às operações que fazem parte da rotina de uma organização. Ou seja, são aquelas operações que acontecem constantemente durante a utilização de um sistema: registro de uma venda, a realização de um pagamento, o cadastro de um cliente, etc. Nesse tipo de ambiente, normalmente existe uma grande quantidade de transações sendo realizadas ao mesmo tempo, porém cada uma delas costuma envolver uma quantidade relativamente pequena de dados.

Já o OLAP possui uma finalidade diferente, pois está muito mais relacionado à análise dos dados e ao apoio na tomada de decisões. Nesse caso, em vez de realizar diversas pequenas alterações nos registros, como acontece no OLTP, o objetivo normalmente é consultar e analisar grandes volumes de informações que foram acumuladas ao longo do tempo. Essas informações podem ser organizadas em estruturas próprias para análise, como os modelos multidimensionais existentes em um *data warehouse*, que recebe dados provenientes de diferentes fontes por meio de processos de extração, tratamento e carga (CHAUDHURI; DAYAL, 1997).

Nesse contexto, os bancos NoSQL podem ser utilizados em ambos os cenários, dependendo do tipo de dado, da aplicação e do padrão de acesso. Em ambientes transacionais, bancos de chave-valor e orientados a documentos podem ser vantajosos em operações simples e de grande volume, como sessões, carrinhos e catálogos, enquanto bancos relacionais continuam sendo mais adequados para situações que exigem maior consistência entre várias entidades, como em operações financeiras (SADALAGE; FOWLER, 2013). No processamento analítico, os bancos NoSQL podem trabalhar com grandes volumes de dados semiestruturados e oferecer recursos de agregação, embora não substituam diretamente os sistemas de *data warehouse* e de armazenamento colunar (STONEBRAKER; PAVLO, 2024).

Na prática, essas tecnologias podem ser utilizadas de maneira conjunta dentro de uma mesma arquitetura. Em uma plataforma de comércio eletrônico, por exemplo, o PostgreSQL pode armazenar pedidos e pagamentos, o Redis pode ser utilizado para sessões e carrinhos, o MongoDB para catálogos com atributos variados, o Cassandra para registros de eventos e o Neo4j para analisar relações entre clientes, dispositivos e transações.

4 COMPARAÇÃO TÉCNICA ENTRE POSTGRESQL E MONGODB

Quadro 1 – Comparação técnica entre o PostgreSQL e o MongoDB

Aspecto	PostgreSQL	MongoDB
Modelo de dados	Relacional (objeto-relacional): tabelas com linhas e colunas tipadas, ligadas por chaves primárias e estrangeiras; admite dados semiestruturados no tipo JSONB.	Orientado a documentos: documentos BSON agrupados em coleções, com subdocumentos e *arrays* aninhados; limite de 16 MB por documento.
Linguagem de consulta	SQL padrão, com junções, subconsultas, funções de janela e operadores próprios para JSON.	MongoDB Query API (MQL): métodos como *find* e *pipeline* de agregação; junções limitadas por meio do estágio $lookup.
Escalabilidade	Predominantemente vertical; réplicas de leitura por replicação física ou lógica e particionamento declarativo; distribuição horizontal da escrita depende de extensões, como o Citus.	Horizontal nativa, por *sharding* (fragmentação dos dados por chave), combinada a *replica sets* com *failover* automático.
Flexibilidade de esquema	Esquema rígido, definido previamente e alterado por comandos DDL (ALTER TABLE); colunas JSONB oferecem flexibilidade pontual.	Esquema flexível: documentos de uma mesma coleção podem ter estruturas distintas; validação opcional por JSON Schema.
Suporte a transações	ACID completo, com controle de concorrência multiversão (MVCC) e níveis de isolamento até o serializável.	Atomicidade por documento; transações ACID multidocumento desde a versão 4.0 (2018), estendidas a *clusters* fragmentados na 4.2, com custo de desempenho.
Consistência e desempenho	Consistência forte; alto desempenho em consultas complexas e junções, apoiado pelo otimizador de consultas e por índices variados (B-tree, GIN, BRIN, entre outros).	Consistência ajustável por *write concern* e *read concern*; alto desempenho em leituras e escritas de documentos completos, sobretudo com dados incorporados.

Foram selecionados o PostgreSQL, SGBD objeto-relacional de código aberto reconhecido pela aderência ao padrão SQL e pela robustez transacional, e o MongoDB, banco orientado a documentos e um dos mais difundidos entre os sistemas NoSQL (MONIRUZZAMAN; HOSSAIN, 2013). O Quadro 1 sintetiza a comparação a partir dos seis aspectos propostos, com base na documentação oficial dos dois sistemas.

A principal diferença entre PostgreSQL e MongoDB está na forma como os dados são organizados. No PostgreSQL, as informações são normalmente distribuídas em tabelas relacionadas, buscando reduzir redundâncias e garantir a integridade dos dados por meio de chaves e relacionamentos, embora isso possa exigir o uso de junções em consultas mais complexas. Já no MongoDB, os dados que costumam ser acessados em conjunto podem ser armazenados dentro de um mesmo documento, o que facilita determinadas consultas e pode melhorar o desempenho, mas também pode gerar duplicação de informações e maior preocupação com a consistência (SADALAGE; FOWLER, 2013). Essa diferença também aparece na linguagem de consulta e na estrutura do esquema, uma vez que o PostgreSQL utiliza

SQL e possui um esquema mais rígido, enquanto o MongoDB oferece maior flexibilidade na estrutura dos documentos.

Também existem diferenças importantes em relação à escalabilidade e às transações. O MongoDB foi desenvolvido com maior foco na distribuição horizontal dos dados, utilizando recursos como *sharding* e permitindo ajustar aspectos relacionados à consistência por meio de *write concern* e *read concern* (MONGODB, 2026). O PostgreSQL, por outro lado, mantém maior ênfase nas garantias ACID e na consistência dos dados, sendo tradicionalmente associado à escalabilidade vertical e ao uso de réplicas de leitura (POSTGRESQL GLOBAL DEVELOPMENT GROUP, 2026).

5 CONCLUSÃO

A partir da análise realizada, é possível perceber que os bancos de dados NoSQL surgem como uma alternativa importante para situações em que os modelos relacionais apresentam limitações, principalmente quando existe a necessidade de trabalhar com grandes volumes de dados, estruturas mais flexíveis e sistemas distribuídos. Entre suas principais vantagens estão a escalabilidade horizontal, a flexibilidade de esquema e o bom desempenho em determinados padrões de acesso. No entanto, essas características também envolvem algumas concessões, principalmente relacionadas à padronização das consultas, à duplicação de dados e à consistência. Dessa forma, a escolha de um banco NoSQL não deve ser feita apenas pelo fato de ele ser mais recente ou mais flexível, mas considerando as características da aplicação e a forma como os dados serão utilizados.

A comparação entre PostgreSQL e MongoDB mostra que não existe necessariamente um modelo melhor para todas as situações. Enquanto o PostgreSQL apresenta vantagens em cenários que exigem maior controle da integridade, consistência e garantias transacionais, o MongoDB pode ser mais adequado quando existe necessidade de maior flexibilidade na estrutura dos dados e facilidade de distribuição. Além disso, os próprios sistemas têm incorporado características uns dos outros, mostrando que a separação entre bancos relacionais e NoSQL não é tão rígida quanto pode parecer.

6 REFERÊNCIAS BIBLIOGRÁFICAS

ELMASRI, R.; NAVATHE, S. B. **Sistemas de banco de dados**. 7. ed. São Paulo: Pearson Education do Brasil, 2018.

HILBERT, Martin; LÓPEZ, Priscila. The world's technological capacity to store, communicate, and compute information. *Science*, v. 332, n. 6025, p. 60-65, 2011. DOI: 10.1126/science.1200970.

CHAUDHURI, S.; DAYAL, U. An overview of data warehousing and OLAP technology. **ACM SIGMOD Record**, v. 26, n. 1, p. 65-74, mar. 1997.

SADALAGE, P. J.; FOWLER, M. **NoSQL essencial**: um guia conciso para o mundo emergente da persistência poliglota. Tradução: Acauan Fernandes. São Paulo: Novatec, 2013.

MONIRUZZAMAN, A. B. M.; HOSSAIN, S. A. NoSQL database: new era of databases for big data analytics – classification, characteristics and comparison. **International Journal of Database Theory and Application**, v. 6, n. 4, p. 1-14, 2013.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 18 documentation**. [S. l.], 2026. Disponível em: https://www.postgresql.org/docs/18/. Acesso em: 26 set. 2026.